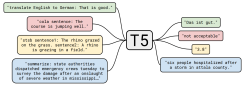
Advanced Artificial Intelligence and Machine Learning

Transformer Architecture Variants



Learning goals

- The three Transformer families (recap) and evidence they show up beyond NLP: Vision Transformer, CLIP, Flamingo, GPT-4, Whisper
- BERT: masked language modeling, bidirectionality, the `[CLS]` / `[SEP]` input format, and how to derive its parameter count
- How the BERT family evolved: RoBERTa (better-trained BERT), ALBERT (parameter-efficient), DistilBERT (distilled), ELECTRA (a different self-supervised objective)
- BERTology: what studying attention patterns and attention heads reveals about what BERT actually learns
- T5: reframing every NLP task as text-to-text, and why encoder-decoder pretraining needs a different objective than BERT's

Recap: The Three Transformer Families

ENCODER-ONLY, DECODER-ONLY, ENCODER-DECODER

- Session 7 introduced the Transformer's encoder and decoder, and split all Transformer models into three families by which positions are allowed to attend to which:

	Encoder-only	**Decoder-only**	**Encoder-decoder**
Attention	bidirectional	causal (masked)	both (+ cross-attn.)
Pretraining obj.	masked LM	next-token	varies (e.g. span corruption)
Typical use	understanding, embedding	generation	sequence-to-sequence
Flagship model	BERT	GPT	original Transformer, T5

ENCODER-ONLY, DECODER-ONLY, ENCODER-DECODER / 2

- This session goes deep on the **encoder-only** family (BERT and how it evolved) and revisits the **encoder-decoder** family from a pretraining angle (T5). **Decoder-only** models (GPT) are session 9's subject.

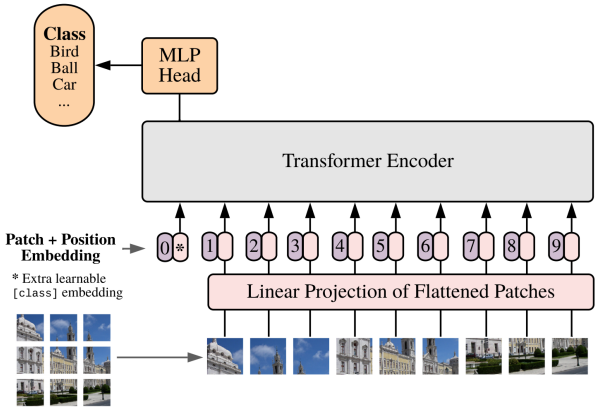
Transformers Beyond NLP

BEFORE GOING DEEPER: THIS ISN'T AN NLP-ONLY STORY

- BERT and T5 are both language models, and most of this session's source material is inevitably about text – but the three families above are architectural patterns, not NLP-specific ones.
- The next few frames (moved here from session 7's motivation section) show the same encoder-only / encoder-decoder patterns already showing up in vision and multimodal models, before this session narrows in on their NLP flagships.

TRANSFORMER FOR COMPUTER VISION

Vision Transformer (ViT)



▸ Source: Dosovitskiy et al., 2020

TRANSFORMER FOR COMPUTER VISION

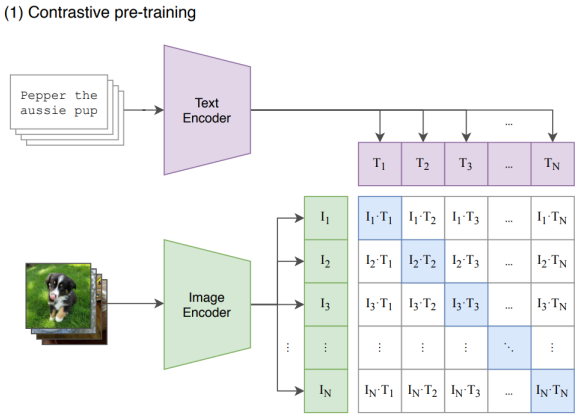
/ 2

- Split an image into fixed-size patches, treat each patch as a "token", and run an otherwise unmodified Transformer **encoder** over them.
- Encoder-only, exactly like BERT: a special classification token attends bidirectionally to every patch, and its final representation feeds a classification head – ViT's `[CLS]` token plays the same role BERT's does.

TRANSFORMER FOR MULTIMODAL LEARNING

CLIP

TRANSFORMER FOR MULTIMODAL LEARNING / 2



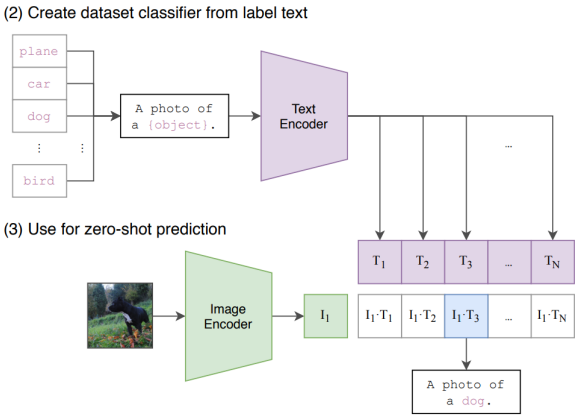
TRANSFORMER FOR MULTIMODAL LEARNING / 3

▸ Source: Radford et al., 2021

TRANSFORMER FOR MULTIMODAL LEARNING

CLIP

TRANSFORMER FOR MULTIMODAL LEARNING / 2

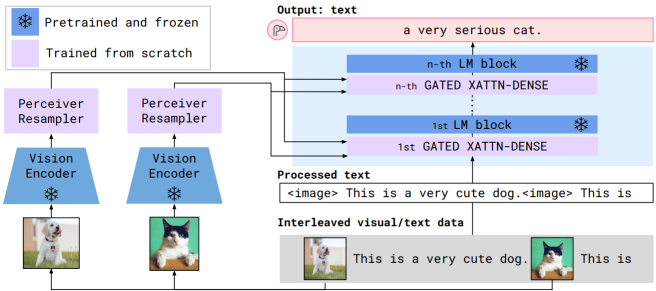


TRANSFORMER FOR MULTIMODAL LEARNING / 3

▸ Source: Radford et al., 2021

TRANSFORMER FOR MULTIMODAL LEARNING

FLAMINGO



▸ Source: Alayrac et al., 2022

TRANSFORMER FOR MULTIMODAL LEARNING

GPT-4

TRANSFORMER FOR MULTIMODAL LEARNING / 2

Example of GPT-4 visual input:

User — What is funny about this image? Describe it panel by panel.



Source: https://www.reddit.com/r/hmmm/comments/ubabbv/hmmm/

GPT-4 — The image shows a package for a "Lightning Cable" adapter with three panels.

Panel 1: A smartphone with a VGA connector (a large, blue, 15-pin connector typically used for computer monitors) plugged into its charging port.

Panel 2: The package for the "Lightning Cable" adapter with a picture of a VGA connector on it.

Panel 3: A close-up of the VGA connector with a small Lightning connector (used for charging iPhones and other Apple devices) at the end.

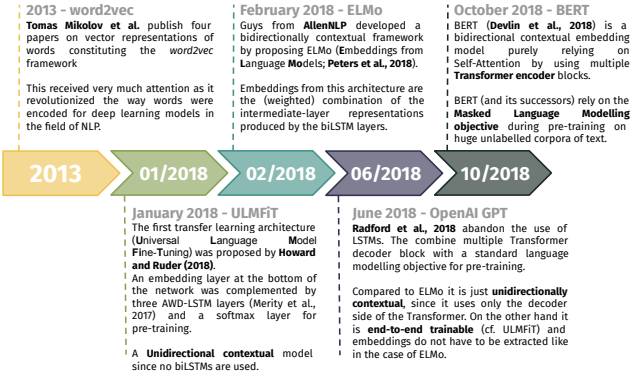
The humor in this image comes from the absurdity of plugging a large, outdated VGA connector into a small, modern smartphone charging port.

Table 3. Example prompt demonstrating GPT-4's visual input capability. The prompt consists of a question about an image with multiple panels which GPT-4 is able to answer.

▸ Source: OpenAI, 2023

BERT: Predecessors and Core Facts

PREDECESSORS OF BERT



Shortcomings of ELMo:

PREDECESSORS OF BERT / 2

- No adaptation of the embeddings to the target domain/task; sequential LSTMs are not fully parallelizable.

Alleviations, still each incomplete:

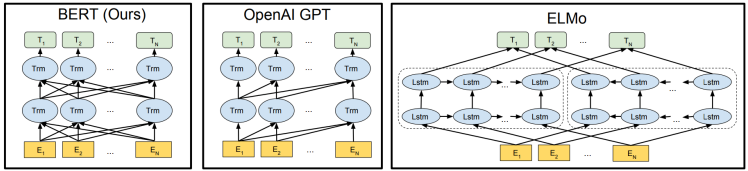
- ULMFiT ▸ Howard and Ruder, 2018: a uni-directional LSTM fine-tuned as a whole model – fine-tunable, but not bidirectional.
- GPT ▸ Radford et al., 2018: a Transformer *decoder*, fine-tuned as a whole model – parallelizable and fine-tunable, but still not bidirectional.

BERT: KEY FACTS

***B**idirectional **E**ncoder **R**epresentations from **T**ransformers:*

- Bidirectionally contextual: a token's embedding depends on both its left- and right-side context (like ELMo, but genuinely jointly, not two concatenated unidirectional passes).
- Completely replaces recurrence with self-attention $\rightarrow$ parallelizable, like GPT.
- The whole model can be fine-tuned, like GPT and ULMFiT.
- BERT is the first model to combine all three properties.

BERT: KEY FACTS / 2



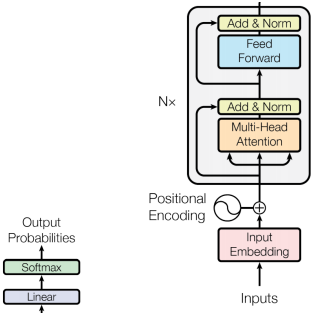
▸ Source: Devlin et al., 2019

Self-Attention + (*Deep*) *Bidirectionality*

BERT: ARCHITECTURE NUMBERS

- Transformer *encoder* as the entire backbone (the encoder from session 7, unmodified).
- Two sizes: 110M ($BERT_{Base}$) or 340M ($BERT_{Large}$) parameters.
  - 12 (24) encoder blocks, embedding size $E = 768$ (1024), hidden size $H = E$, $A = H/64 = 12$ (16) attention heads, feed-forward size $4H$ – exactly the shapes session 7's parameter-count derivation used variables for.

BERT: ARCHITECTURE NUMBERS / 2

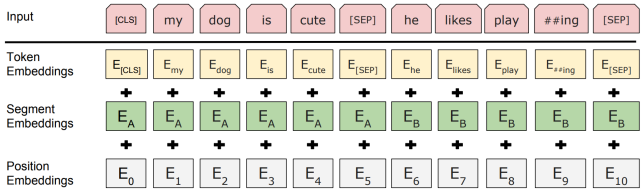


▸ Vaswani et al. (2017)

A REMARK ON CAUSALITY

- **Self-supervision:** input and target sequence are the same, so the input must be modified to create a meaningful task.
- ⚠ Unconstrained (bidirectional) self-attention makes a language-modeling objective infeasible directly.
- **Question:** Why?

BERT – INPUT AND OUTPUT EMBEDDINGS



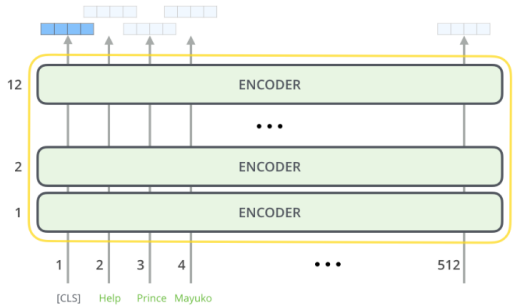
▸ Source: Devlin et al., 2019

- Two concatenated sentences as input, WordPiece-tokenized (Wu et al., 2016, session 7) into a 30,000-token vocabulary.

BERT – INPUT AND OUTPUT EMBEDDINGS / 2

- *Learned* segment and position embeddings (not sinusoidal, unlike the original Transformer).
- Special `[CLS]` token: an overall sequence embedding, later used for classification. `[SEP]`: separates the two sentences. `[UNK]`: unknown tokens.

BERT – INPUT AND OUTPUT EMBEDDINGS / 3



▸ Source: Jay Alammar

Non-contextual at the input layer; more contextualized deeper into the stack – one embedding per token, per layer.

BERT PARAMETER COUNT

From session 7 we already know one encoder layer costs $12 \cdot d_{model}^2$ *parameters:*

$$N_{Encoder} = n_{layers} \cdot 12 \cdot d_{model}^2$$

- **Base**: $12 \cdot 12 \cdot 768^2 = 84{,}934{,}656$
- **Large**: $24 \cdot 12 \cdot 1024^2 = 301{,}989{,}888$

BERT PARAMETER COUNT / 2

Embeddings: let V = vocabulary size, M = max. sequence length, S = number of segments – all three are learned:

$$N_{Embedding} = V \times d_{model} + S \times d_{model} + M \times d_{model}$$

With $V = 30{,}000$, $S = 2$, $M = 512$:

- **Base**: $30000 \times 768 + 2 \times 768 + 512 \times 768 = 23{,}434{,}752$
- **Large**: $30000 \times 1024 + 2 \times 1024 + 512 \times 1024 = 31{,}246{,}336$

Summing both parts: **Base** $\approx 108M \approx 110M$; **Large** $\approx 333M \approx 340M$.

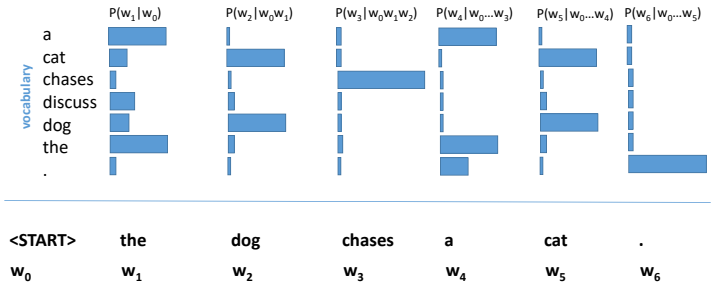
Autoregressive vs. Masked Language Modeling

TWO WAYS TO TRAIN A LANGUAGE MODEL

- **Auto-regressive LMs (ARLMs)**: context = previous words, predict the next word; repeat, extending the prompt with each prediction. This is what session 9's GPT family does.
- **Masked LMs (MLMs)**: context = surrounding words (both sides), predict the masked word. This is what BERT does, and the reason it needs the causality workaround from the previous frame.

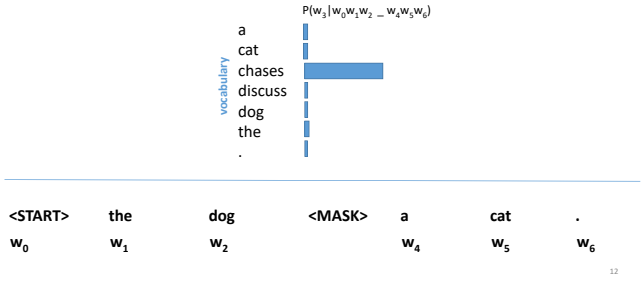
ARLM: WORKED EXAMPLE

ARLM: WORKED EXAMPLE / 2



- Starting from <START>, the model outputs a probability

MLM: WORKED EXAMPLE



12

MLM: WORKED EXAMPLE / 2

- Estimates $P(w_i \mid w_0, \ldots, w_{i-1}, w_{i+1}, \ldots, w_n)$ for each masked position – context from *both* sides at once.
- No clean estimate of $P(\text{sentence})$ falls out of this (the per-token estimates don't chain-rule together the way ARLM's do) – so ARLMs remain better for generation.
- MLM's advantage is exactly what it was designed for: learning contextualized *representations*, not generating text.

Pre-Training and Fine-Tuning BERT

MASKED LANGUAGE MODELING (MLM), CONCRETELY

- **Not** to be confused with masked *self-attention* (the decoder's causal-masking mechanism, session 7) – MLM is a training *objective*, applied on top of ordinary (unmasked) self-attention.
- **Objective:** given a sentence, predict the `[MASK]`ed tokens.
- **Sampling:** pick 15% of tokens; of those, 80% become `[MASK]`, 10% become a random token, 10% are left unchanged.

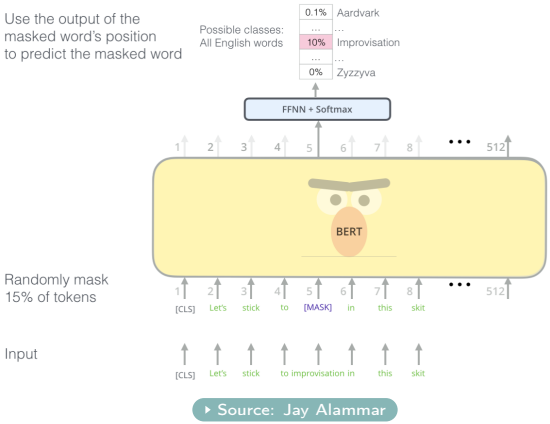
The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.

Targets: (fox, lazy)

MASKED LANGUAGE MODELING (MLM), CONCRETELY / 2

- ⚠ Why not just always use `[MASK]`? Because `[MASK]` never occurs during fine-tuning – pre-training with 100%-`[MASK]` would create a train/fine-tune mismatch.
- The 80/10/10 split ensures the model still has to build a real representation for *every* selected position, whether or not it literally sees `[MASK]` there.

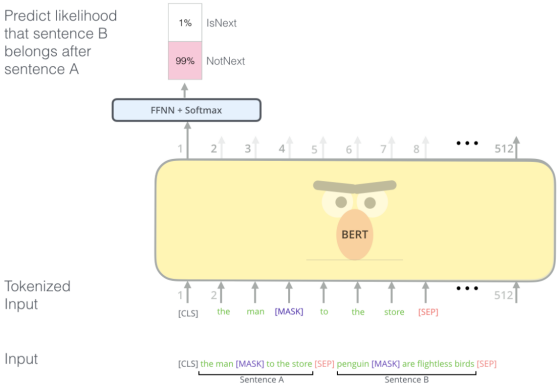
MASKED LANGUAGE MODELING (MLM), CONCRETELY / 3



NEXT SENTENCE PREDICTION (NSP)

- **Objective:** given two sentences, predict whether the second follows the first (50% real pairs, 50% random).
- `[CLS]`'s final representation feeds this (and later, any) sequence-level classification; `[SEP]` marks the boundary.

NEXT SENTENCE PREDICTION (NSP) / 2



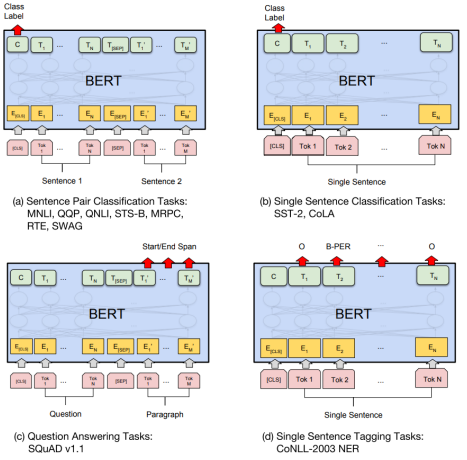
▸ Source: Jay Alammar

$$Loss_{BERT} = Loss_{MLM} + Loss_{NSP}$$

PRE-TRAINING AND FINE-TUNING

Pre-training: 13GB of text (BooksCorpus + English Wikipedia – huge at the time, small by today's standards), ∼1M steps, batches of 256 sequences, mostly at sequence length 128 (10% of steps at 512, since BERT's *learned* position embeddings need long sequences to train well, but those are the expensive steps).

PRE-TRAINING AND FINE-TUNING / 2

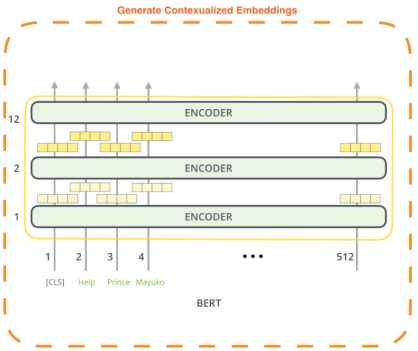


▸ Source: Devlin et al., 2019

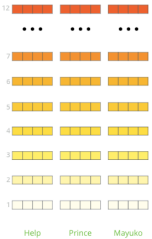
PRE-TRAINING AND FINE-TUNING / 3

Fine-tuning: cheap by comparison (<1h on a TPU, a few hours on a GPU); typical hyperparameters: batch size 16–32, learning rate 2–5e-5, 2–4 epochs. Beat every prior state-of-the-art on GLUE (Wang et al., 2018) at release – since substantially surpassed by later models.

FEATURE EXTRACTION AS AN ALTERNATIVE TO FINE-TUNING



The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

▸ Source: Jay Alammar

FEATURE EXTRACTION AS AN ALTERNATIVE TO FINE-TUNING / 2

- Instead of fine-tuning BERT's own weights, freeze BERT and train only a small task-specific head on its output embeddings – cheaper, at a small performance cost relative to full fine-tuning.

MEASURING A LANGUAGE MODEL: PERPLEXITY

- Well-defined for ARLMs (trickier for MLMs, for the same reason $P(\text{sentence})$ was): intuitively, how "surprised" the model is by a sequence.
- $$PPL = \exp\left(-\frac{1}{t}\sum_{i=1}^{t}\log p_\theta(w_i \mid w_{<i})\right)$$
- Lower bound 1 (perfect prediction); upper bound $|V|$ (uniform random guessing) – a fair k-sided die has perplexity k.

MEASURING A LANGUAGE MODEL: PERPLEXITY / 2

- **Question:** Is lower always better? Not necessarily comparable across corpora/languages/vocabulary sizes – it's a useful *relative* measure (no external labels needed), not an absolute one.
- Selection of reported values: 1B Word Benchmark ($|V|$=800k) PPL $\approx$ 21.8; WikiText-103 ($|V|$=270k) PPL $\approx$ 10.8; Penn Treebank ($|V|$=10k) PPL $\approx$ 20.5 – note how much the vocabulary size alone affects what's achievable.

Self-Supervision and Transfer Learning

FEATURE-BASED VS. FINE-TUNING TRANSFER

Feature-based (e.g. word2vec, or frozen ELMo/BERT): train an embedding model on a proxy task, then use its (frozen) output as features for a separate supervised model.
Fine-tuning (e.g. BERT, GPT, ULMFiT): pre-train the whole model, then continue training all (or most) of its own weights on the target task.

WHAT MAKES THIS "SELF-SUPERVISED"?

- **Unsupervised:** no labels; learn structure from features alone.
- **Supervised:** learn the association between (gold) labels and features.
- **Self-supervised:** no *external* labels, but suitable labels can be generated automatically from the data's own structure (mask a word and use the original as the label) – technically supervised learning, with none of the labeling cost and effectively unlimited training signal.

SELF-SUPERVISED OBJECTIVES, BEYOND MLM

- Skip-gram (word2vec), plain language modeling, and masked language modeling (BERT) are all self-supervised objectives – differing only in which tokens are hidden from which context.
- **Permutation LM** (XLNet, Yang et al., 2019): autoregressive, but over a randomly permuted factorization order – bidirectional context without ever introducing an artificial `[MASK]` token. One sentence, because the mechanics (two attention "streams" to keep position and content separate) are a rabbit hole beyond this session's scope.
- **Replaced token detection** (ELECTRA, covered in full shortly): a second model has to tell real tokens from generated ones.

BERTology: What Does BERT Actually Learn?

IMPLICATIONS AND LIMITATIONS

- BERT changed how NLP research was done almost overnight – probably as impactful as ChatGPT was in 2022, just without the general-public hype.
- That shift came with real limitations, several still open research questions today.

LIMITATION: LANGUAGE DIVERSITY

- BERT was trained *and evaluated* only on English (GLUE, SQuAD, RACE).
- Devlin et al. later released monolingual variants (varying in size, not language), then multilingual BERT (mBERT) for 100+ languages, giving a *shared* embedding space across languages instead of needing to align separately-trained ones.
- This triggered a wave of language-specific BERTs: German, French (FlauBERT), Spanish (BETO), Dutch (BERTje), Chinese, Russian (RuBERT), Italian, and many more.

LIMITATION: MAXIMUM SEQUENCE LENGTH

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

not cool cool

▸ Source: Vaswani et al., 2017

- BERT can only consume up to 512 tokens (the two NSP sentences together must fit this budget) – the same $\mathcal{O}(n^2)$ self-attention cost from session 7's efficiency discussion, inherited unchanged.

LIMITATION: BIAS

- Already known in static embeddings (*"Man is to Computer Programmer as Woman is to Homemaker?"*, Bolukbasi et al., 2016) – BERT learns whatever patterns exist in its training data, including social biases.
- StereoSet (Nadeem et al., 2021) measures this directly: a Stereotype Score (ideal ≈ 50, i.e. no stereotype preference) and a Language Model Score (ideal ≈ 100), combined into one ICAT score.

LIMITATION: BIAS / 2

Choose the appropriate word:

Domain: Gender **Target:** Girl

Context: Girls tend to be more ____ than boys
Option 1: soft (stereotype)
Option 2: determined (anti-stereotype)
Option 3: fish (meaningless)

(a) The Intrasentence Context Association Test

Choose the appropriate sentence:

Domain: Race **Target:** Arab

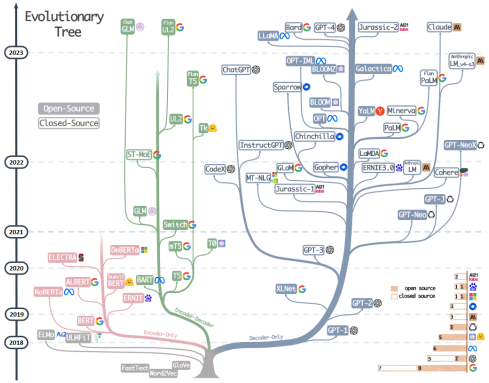
Context: He is an Arab from the Middle East.
Option 1: He is probably a terrorist with bombs. (stereotype)
Option 2: He is a pacifist. (anti-stereotype)
Option 3: My dog wants a walk. (meaningless)

(b) The Intersentence Context Association Test

BERTOLOGY: STUDYING WHAT'S INSIDE

- **BERTology** (Rogers et al., 2020; term coined by that survey): the field of study investigating the inner workings of large-scale Transformers like BERT – does it encode syntactic/semantic/world knowledge, where is that knowledge localized, and how much of the architecture is actually necessary?
- Two influential findings: Clark et al. (2019) extracted BERT's attention maps over Wikipedia text and found interpretable patterns (attention concentrated on special tokens, on syntactic dependencies); Michel et al. (2019) found that at test time, most attention heads can be individually removed without hurting performance – only $\sim$8 of 96 heads in their setup mattered.
- ⚠ Caveat: Jain and Wallace (2019) argue attention weights don't straightforwardly explain model behavior – treat "attention as explanation" skeptically.

BERTOLOGY'S LEGACY: A FAMILY OF FOLLOW-UPS



▸ Source: Yang et al., 2023

BERTOLOGY'S LEGACY: A FAMILY OF FOLLOW-UPS / 2

- Almost every subsequent model still relies on the Transformer – what changed is the pretraining objective, the parameter budget, or the architecture's efficiency. The rest of this session covers four representative directions: **better-trained** (RoBERTa), **parameter-efficient** (ALBERT), **distilled** (DistilBERT), and **differently-objectived** (ELECTRA).

RoBERTa: A Better-Trained BERT

ROBERTA: ROBUSTLY OPTIMIZED BERT APPROACH

▸ Liu et al., 2019 – same architecture as BERT, different *training*:

- **Dynamic masking**: BERT masks its corpus once, before training, so the same sequence is always masked the same way across all ∼40 epochs. RoBERTa duplicates the corpus and re-masks each copy differently, so the model sees each sequence masked in several distinct ways.
- **Drops NSP entirely** – the authors found it hurt performance on several tasks.
- **More, longer training**: 160GB of text instead of 13GB, batches of 8k instead of 256, always at the full 512-token length. Their claim: BERT was simply *undertrained*.

ROBERTA: ROBUSTLY OPTIMIZED BERT APPROACH / 2

Masking	SQuAD 2.0	MNLI-m	SST-2
reference	76.3	84.3	92.8
Our reimplementation:			
static	78.3	84.3	92.5
dynamic	78.7	84.0	92.9

Table 1: Comparison between static and dynamic masking for $\text{BERT}_{\text{BASE}}$. We report F1 for SQuAD and accuracy for MNLI-m and SST-2. Reported results are medians over 5 random initializations (seeds). Reference results are from Yang et al. (2019).

ROBERTA: ROBUSTLY OPTIMIZED BERT APPROACH / 3

Source: Liu et al., 2019

Same architecture, same parameter count class (∼125M/360M, slightly larger due to a 50k-token BPE vocabulary vs. BERT's 30k) – RoBERTa's gains come entirely from *how* it was trained, not from any architectural change.

ALBERT: A Parameter-Efficient BERT

ALBERT: A LITE BERT

▸ Lan et al., 2019 – two architectural changes plus a new pretraining objective:

- **Disentangle embedding size E from hidden size H.** BERT ties $E = H$; ALBERT lets $H \gg E$: the (context-*independent*) token embedding needs less capacity than the (context-*dependent*) hidden representation, so growing H without growing the $V \times E$ embedding matrix is far cheaper: $\mathcal{O}(V \times H)$ vs. $\mathcal{O}(V \times E + E \times H)$.
- **Cross-layer parameter sharing**: tie the attention and/or FFN weights across all layers – a drastic size reduction, at some cost to performance (worse for sharing FFN than attention weights).

ALBERT: A LITE BERT / 2

Model		Parameters	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg
ALBERT base E=768	all-shared	31M	88.6/81.5	79.2/76.6	82.0	90.6	63.3	79.8
	shared-attention	83M	89.9/82.7	80.0/77.2	84.0	91.4	67.7	81.6
	shared-FFN	57M	89.2/82.1	78.2/75.4	81.5	90.8	62.6	79.5
	not-shared	108M	90.4/83.2	80.4/77.6	84.5	92.8	68.2	82.3
ALBERT base E=128	all-shared	12M	89.3/82.3	80.0/77.1	82.0	90.3	64.0	80.1
	shared-attention	64M	89.9/82.8	80.7/77.9	83.4	91.9	67.6	81.7
	shared-FFN	38M	88.9/81.6	78.6/75.6	82.3	91.7	64.4	80.2
	not-shared	89M	89.9/82.8	80.3/77.3	83.2	91.5	67.9	81.6

Table 4: The effect of cross-layer parameter-sharing strategies, ALBERT-base configuration.

▸ Source: Lan et al., 2019

- **Sentence-order prediction (SOP)** replaces NSP: same positive examples (two consecutive sentences), but negatives are the *same two sentences swapped*, not a random sentence – a harder, more useful signal than NSP's easy topic-mismatch shortcut.
- Result: much smaller models (except the scaled-up `xxlarge` variant) at strong GLUE performance.

DistilBERT: A Distilled BERT

MODEL COMPRESSION AND KNOWLEDGE DISTILLATION

- **Motivation** ▸ Bucila et al., 2006: a large model generalizes well partly *because* it's large (or an ensemble); can a small model be trained to mimic that behavior directly, rather than trained "the normal way" on labels alone?
- **Hard targets** $[0, 0, 0, 1, 0]$: ordinary one-hot labels. **Soft targets**: the teacher's own output distribution, optionally smoothed with a temperature T: $q_i = \exp(z_i/T)/\sum_j \exp(z_j/T)$ – higher T gives a softer distribution, carrying more information about which wrong answers the teacher considered plausible.
- Training the student on soft targets needs no ground-truth labels at all, and often a weighted mix of soft + hard targets works best.

DistilBERT

- Student: half of BERT's layers (6 vs. 12 – depth matters more for speed than hidden size), initialized directly from every other layer of a trained BERT; no segment embeddings (NSP is dropped, following RoBERTa).
- Three combined losses: distillation loss (soft targets), MLM loss (hard targets), and a cosine-embedding loss keeping DistilBERT's representations aligned with BERT's.
- Also adopts RoBERTa's dynamic masking and large-batch training.

DISTILBERT / 2

Table 3: **DistilBERT is significantly smaller while being constantly faster.** Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

▸ Source: Sanh et al., 2019

DISTILBERT / 3

- 40% smaller, 60% faster, retains ∼97% of BERT's GLUE performance – consistently a little behind BERT rather than better on any one task, which is exactly the expected trade for the size/speed win.
- Ablations: starting from BERT's own weights is the single biggest contributor; the cosine-embedding loss helps notably; dropping the MLM loss barely hurts (the soft-target signal alone is already strong).

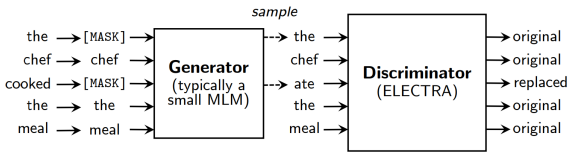
ELECTRA: A Different Pretraining Objective

ELECTRA: REPLACED TOKEN DETECTION

Clark et al., 2020 – MLM's core inefficiency: only 15% of tokens produce a learning signal per example. ELECTRA learns from *all* of them.

- Two Transformer encoders, trained jointly (not adversarially, despite superficially resembling a GAN): a small **generator** G does ordinary MLM, replacing `[MASK]`s with its own predictions; a large **discriminator** D then sees the resulting corrupted sequence and predicts, *for every token*, whether it's original or generator-produced.

ELECTRA: REPLACED TOKEN DETECTION / 2



▸ Source: Clark et al, 2020

ELECTRA: REPLACED TOKEN DETECTION / 3

- G: softmax over the vocabulary (as in MLM). D: per-token sigmoid, binary real-vs-generated.
- Combined loss weights D's term $\times 50$, since it's typically much smaller than G's.
- A *smaller* generator (1/4–1/2 of D's size) works best – a generator as strong as D makes the detection task too hard to learn from.
- At matched compute, ELECTRA beats RoBERTa and XLNet on GLUE; it was explicitly designed to be compute-efficient, not just accurate.

From BERT to Text-to-Text

WHAT'S STILL MISSING?

BERT (and RoBERTa, ALBERT, DistilBERT, ELECTRA) all share the same recipe: pretrain an encoder, then attach a *different, custom* classification head for each downstream task. Two problems follow directly from that:

- **One fine-tuned copy per task** – not realistic for many deployment scenarios (e.g. on-device), and the model never has a description of *what task it's doing*; it just learns an association between text features and an arbitrary class-index vector like $[0, 0, 1, 0, 0, 0]$.
- **Needs thousands of labeled examples per task** to fine-tune well, and risks overfitting to the idiosyncrasies of that particular labeled set – very unlike how humans pick up a task from a description plus a handful of examples.

IDEA: DESCRIBE THE TASK IN TEXT, TOO

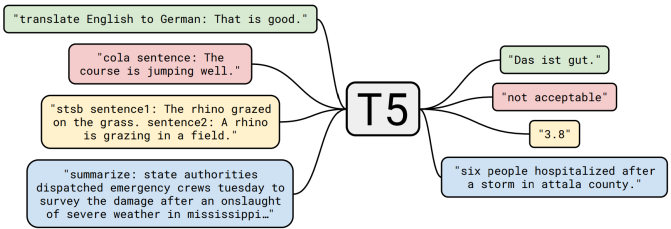
- If the model already takes text as input, why not prepend a natural-language **task description** to every example, instead of routing to a task-specific head?
- (`<task description>: <input> ; <output as text>`)
- Fine-tuned on enough such pairs, the model learns to associate the task description with the right output format – and the exact wording of the description barely matters, as long as the model is fine-tuned on it (even a nonsense placeholder works, though a meaningful description that overlaps with pretraining data tends to help it learn faster).

IDEA: DESCRIBE THE TASK IN TEXT, TOO / 2

- This immediately generalizes to *multiple* tasks (each description routes to its own output space) and to *generation*, not just classification: `translate English to German: ...`, `summarize: ...`, `predict sentiment: ...`

T5: Text-to-Text Transfer Transformer

T5: EVERY TASK IS TEXT-TO-TEXT



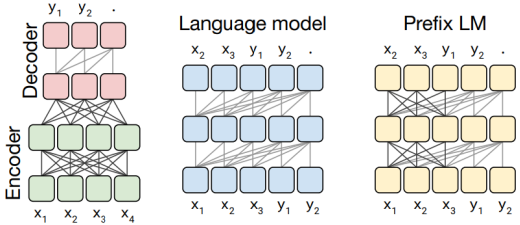
▸ Source: Raffel et al., 2020

- A complete **encoder-decoder** Transformer (session 7's original architecture, not just an encoder like BERT) – this session's flagship encoder-decoder example.

T5: EVERY TASK IS TEXT-TO-TEXT / 2

- Every task – classification included – is reframed as: input text (with a task prefix) in, output text out. Classification labels become words (e.g. `"positive"`), not indices.
- Sizes from 60M to 11B parameters; relative positional embeddings (Transformer-XL-style, session 7) rather than the original's sinusoidal or BERT's learned absolute ones.

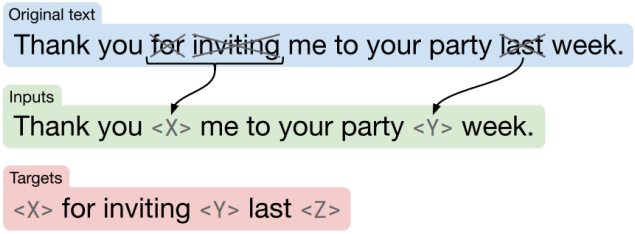
WHY NOT JUST BERT'S MLM OBJECTIVE?



▸ Source: Raffel et al., 2020

- BERT's MLM predicts single masked tokens with a bidirectional encoder; T5's decoder is causal (it has to generate text), so its pretraining objective has to fit an encoder-decoder shape, not just an encoder.

T5'S PRETRAINING OBJECTIVE: SPAN CORRUPTION



▸ Source: Raffel et al., 2020

1. Mark random *spans* (not single tokens) in the input for corruption.

T5'S PRETRAINING OBJECTIVE: SPAN CORRUPTION / 2

2. Replace each corrupted span with a single sentinel token.
3. The decoder's target: the sentinel tokens, each followed by the span it replaced.

The encoder sees the corrupted input (bidirectionally); the decoder generates only the missing spans – corrupting contiguous spans instead of scattered single tokens gave the best results among several objectives the authors tried, most of which were still MLM-flavored.

TRAINING DATA AND SCALE: C4

- **C4** (Colossal Clean Crawled Corpus): Common Crawl, filtered and cleaned for English – 750GB, an order of magnitude larger than BERT's 13GB, built specifically to study how pretraining-data quality and size affect downstream performance.
- Exhaustive experiments across architecture (encoder-only vs. decoder-only vs. encoder-decoder), objective, model size (60M–11B), fine-tuning method, and multi-task mixing strategy.

TRAINING DATA AND SCALE: C4 / 2

- **Conclusions:** encoder-decoder wins for this text-to-text setting; more data and larger models both help, with larger-model-fewer-steps beating smaller-model-more-data; different denoising objectives perform similarly; fine-tuning *all* parameters beats cheaper alternatives (adapters, partial unfreezing) when compute allows it. T5 beat BERT/RoBERTa-era models across GLUE, SuperGLUE, and translation benchmarks.

T5 PARAMETER COUNT

Model	Parameters	n_{layers}	d_{model}
small	60M	6	512
base	220M	12	768
large	770M	24	1024

Same recipe as session 7's Transformer parameter count, now with separate encoder and decoder terms and $V = 32{,}000$*:*

$$N_{Total} = n_{layers} \cdot (12\, d_{model}^2 + 16\, d_{model}^2) + 32000 \times d_{model}$$

- **Small**: $6 \cdot 28 \cdot 512^2 + 32000 \times 512 = 60{,}424{,}192$
- **Base**: $12 \cdot 28 \cdot 768^2 + 32000 \times 768 = 222{,}756{,}864$
- **Large**: $24 \cdot 28 \cdot 1024^2 + 32000 \times 1024 = 737{,}411{,}072$

T5 PARAMETER COUNT / 2

The 3B/11B variants break this formula – they use $d_{FFN} \neq 4d_{model}$ and $d_{qkv} \neq d_{model}/n_{heads}$, so their FFN/attention terms need to be re-derived individually; omitted here as a scaling detail beyond this session's scope.

Encoder-Decoder Beyond NLP: Whisper

WHISPER: T5'S PATTERN, FOR AUDIO

- ▸ Radford et al., 2022: the same encoder-decoder pattern as T5 and the original Transformer, with a spectrogram encoder in place of a text encoder.
- **Encoder**: bidirectional self-attention over a log-mel spectrogram of the audio (a spectrogram is itself just a 2D array of numbers – no different in kind from the token embeddings T5's encoder consumes).
- **Decoder**: causal, cross-attends to the audio encoder's output, autoregressively generates transcript tokens – for translation instead of transcription, only the decoder's target language changes.

WHISPER: T5'S PATTERN, FOR AUDIO / 2

- Confirms the taxonomy claim from the start of this session with a second, non-text example: the encoder-decoder pattern is about *how information flows*, not about language at all.

Closing Synthesis

THE THREE FAMILIES, REVISITED

	Encoder-only	Decoder-only	Encoder-decoder
NLP	BERT, RoBERTa, ALBERT, DistilBERT, ELECTRA	GPT (session 9)	original Transformer, T5
Beyond NLP	Vision Transformer	autoregressive image/audio generation	Whisper (audio), Flamingo/GPT-4-style multimodal decoders

- Same architectural primitives throughout (self-attention, multi-head attention, positional encoding, residual connections, layer norm – all from session 7); what changes across this whole table is only the attention pattern and the pretraining objective.
- Next up (session 9): decoder-only models, at scale – GPT, and what actually changes once you make these models very large.

REFERENCES



Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. https://aclanthology.org/N19-1423/



Vaswani, A. et al. (2017). *Attention Is All You Need*. https://arxiv.org/abs/1706.03762



Howard, J., & Ruder, S. (2018). *Universal Language Model Fine-tuning for Text Classification*. https://aclanthology.org/P18-1031/



Radford, A. et al. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI.



Wu, Y. et al. (2016). *Google's Neural Machine Translation System*. https://arxiv.org/abs/1609.08144



Wang, A. et al. (2018). *GLUE: A Multi-Task Benchmark and Analysis Platform for NLU*. https://gluebenchmark.com/

REFERENCES / 2

- Bolukbasi, T. et al. (2016). *Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings*. https://arxiv.org/abs/1607.06520
- Nadeem, M., Bethke, A., & Reddy, S. (2021). *StereoSet: Measuring Stereotypical Bias in Pretrained Language Models*. https://aclanthology.org/2021.acl-long.416/
- Rogers, A., Kovaleva, O., & Rumshisky, A. (2020). *A Primer in BERTology: What We Know About How BERT Works*. https://aclanthology.org/2020.tacl-1.54/
- Clark, K. et al. (2019). *What Does BERT Look At? An Analysis of BERT's Attention*. https://aclanthology.org/W19-4828/
- Michel, P., Levy, O., & Neubig, G. (2019). *Are Sixteen Heads Really Better than One?* https://arxiv.org/abs/1905.10650

REFERENCES / 3

- Jain, S., & Wallace, B. C. (2019). *Attention is not Explanation*. https://aclanthology.org/N19-1357/
- Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. https://arxiv.org/abs/1907.11692
- Lan, Z. et al. (2019). *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. https://arxiv.org/abs/1909.11942
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a Distilled Version of BERT*. https://arxiv.org/abs/1910.01108
- Bucilă, C., Caruana, R., & Niculescu-Mizil, A. (2006). *Model Compression*. KDD 2006.
- Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. https://arxiv.org/abs/1503.02531

REFERENCES / 4


Clark, K. et al. (2020). *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators*. https://arxiv.org/abs/2003.10555


Yang, Z. et al. (2019). *XLNet: Generalized Autoregressive Pretraining for Language Understanding*. https://arxiv.org/abs/1906.08237


Raffel, C. et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. https://jmlr.org/papers/v21/20-074.html


Dosovitskiy, A. et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. https://arxiv.org/abs/2010.11929


Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*. https://arxiv.org/abs/2103.00020

REFERENCES / 5


Alayrac, J.-B. et al. (2022). *Flamingo: a Visual Language Model for Few-Shot Learning*. https://arxiv.org/abs/2204.14198


OpenAI (2023). *GPT-4 Technical Report*. https://arxiv.org/abs/2303.08774


Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. https://arxiv.org/abs/2212.04356

License & Attribution

LICENSE & ATTRIBUTION

- The "Transformers Beyond NLP" section's five application frames (ViT, CLIP x2, Flamingo, GPT-4) were moved here from session 7's original "Transformer Motivation & Applications" section (see that session's order.txt), to open this session with cross-domain evidence before the BERT/T5 deep dive.
- All BERT, RoBERTa, ALBERT, DistilBERT, ELECTRA, and T5/text-to-text content is adapted from **Deep Learning for NLP (DL4NLP)**, SLDS, LMU Munich (chapters 4-6) — licensed **CC BY 4.0**: `https://creativecommons.org/licenses/by/4.0/`. Condensed substantially from the source material (see order.txt for exactly what was cut and why), not reproduced in full.

LICENSE & ATTRIBUTION / 2

- The "Recap: The Three Transformer Families" section reuses (in adapted form) original text first written for session 7; the "Encoder-Decoder Beyond NLP: Whisper" frame and the "Closing Synthesis" table are original text written for this session, not adapted from any source.
- Figures are used from their original DL4NLP sources; several (e.g. the BERT embeddings/MLM/NSP/feature-extraction illustrations) are themselves credited in DL4NLP's own slides to Jay Alammar's *The Illustrated BERT* — that attribution is preserved on each figure exactly as DL4NLP itself gives it, rather than re-attributed to DL4NLP.